



Advanced and Distributed Algorithms

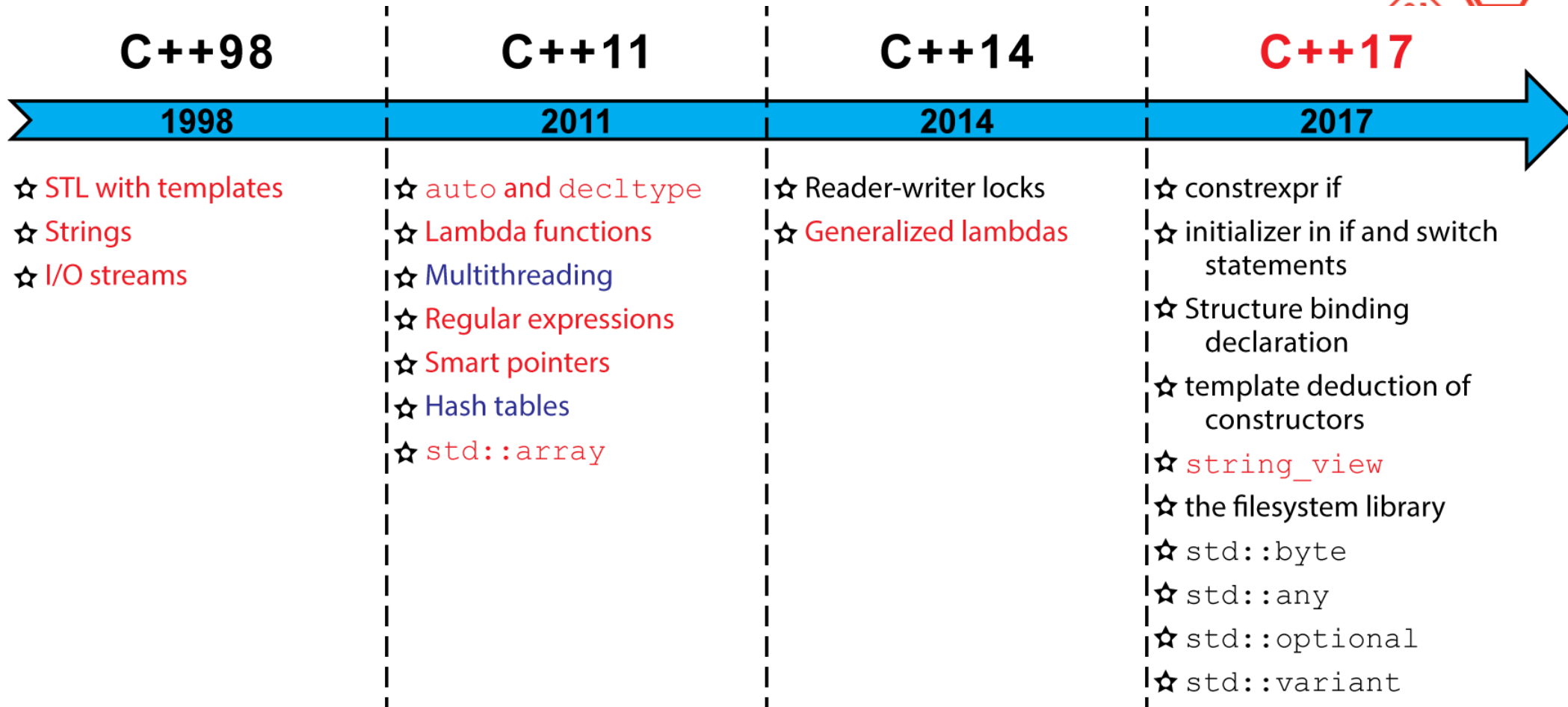
Overview of C++ programming

Pr. Benoît PIRANDA

benoit.piranda@univ-fcomte.fr



C++ versions



Outline



- C++ Basics reminders
 - Class, attributes, methods, operators
 - Inheritance, access control
 - Polymorphism
 - Templates
- STL Library with templates
 - Data structures
 - Lambda function
 - Smart pointers
 - Regular expressions

C++ Basics, data and types



- C++ is a strictly typed language
- Definition : **type identifier [= valeur initiale];**
 - Boolean (tests)
 - Bool (true/false)
 - Integers
 - char (1 byte), short (2 bytes), int (4 bytes), long (8 bytes)
 - Floating-point real numbers
 - float (4 bytes), double (8 bytes)
- **auto**: type set by the initialization value

```
int a=3;  
float b=2.0f;  
double c=4.5;  
bool x=true;
```

```
auto f=1.6f;
```

C++ Basics, data and types



- Fixed width integer (C++11)
 - The size is clearly indicated in the type name
 - Usual in IoT
 - Independent to system, ease debugging
 - Library <cstdint>
 - Signed integers
 - int8_t, int16_t, int32_t, int64_t
 - Unsigned
 - uint8_t, uint16_t, uint32_t, uint64_t

C++ Basics, data and types



- The two operands of an assignment **must be of the same type**.
- Type cast may be done:
 - Manually
 - `int i = int(1.6);`
 - But this **can lead to a loss of information**, at the user's own risk.
 - Automatic when there is no risk of loss of information
 - `float a = 1;`

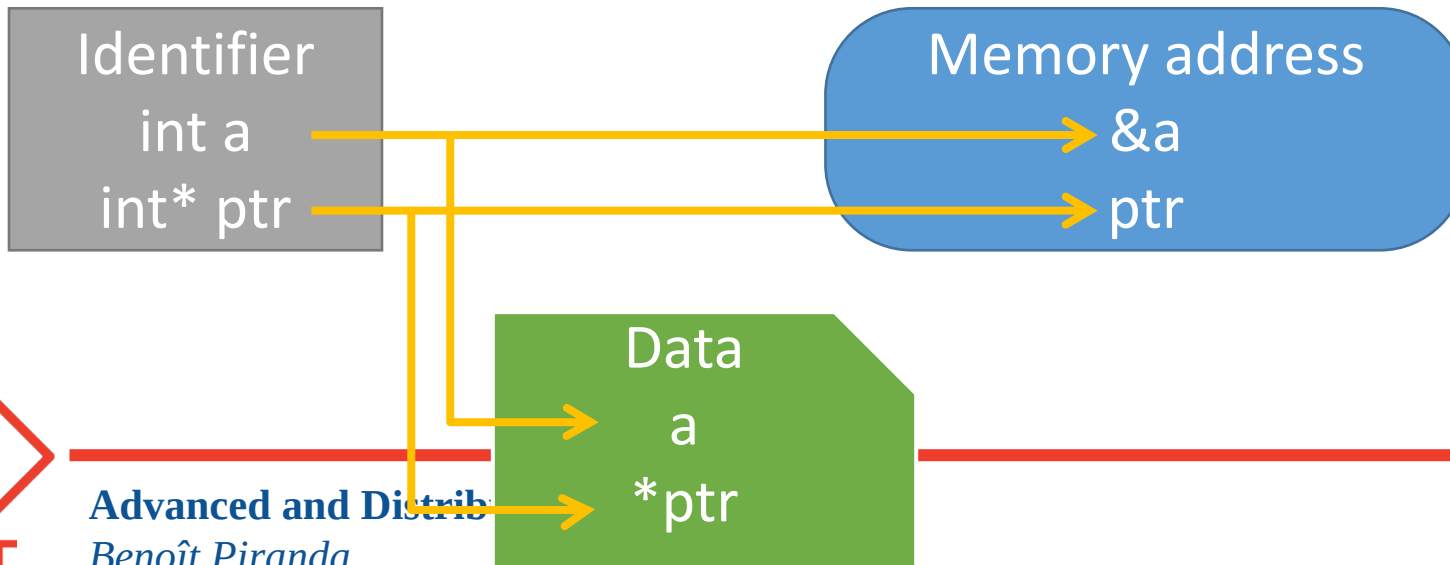
```
int a = 3;  
float b = 2.0f;  
double c = 4.5;  
bool x = true;
```

```
c = b;  
a = int(3.2f);
```

Pointer et addresses (from C)



- When defining a variable, we can distinguish between:
 - The name of the variable (identifier)
 - The memory space (address) where it is stored
 - The value stored in memory



```
#include <iostream>

int main() {
    int a = 3;
    int* ptra = &a;
    int* ptrb;

    std::cout<< a << std::endl;
    *ptra = 5;
    std::cout<< a << std::endl;
    ptrb = &a;
    *ptrb = 6;
    std::cout<< a << std::endl;
}
```

C++ References



- It is also possible **to directly allocate a reference** to another existing variable, or to an element of an array.

```
#include <iostream>

int main() {
    int a=2;
    int& refa=a;
    std::cout << "refa=" << refa << std::endl;
    refa = 3;
    std::cout << "a=" << a << std::endl;

    int taba[5]={2,5,3,4,1};
    refa=taba[4];
    std::cout << "refa=" << refa << std::endl;
}
```

```
refa=2
a=3
refa=1
```

C++ Basics, data and types



- Constants “**const**”

- Identifier whose value cannot be changed (compilation error).
- Type and initial value are mandatory.

```
const int tabSize=100;
```

- **extern** variables

- Variable defined in another project file.
- Initialization forbidden in the second file.

Fichier 1

```
int globalSize=1000;
```

Fichier 2

```
extern int globalSize;
```

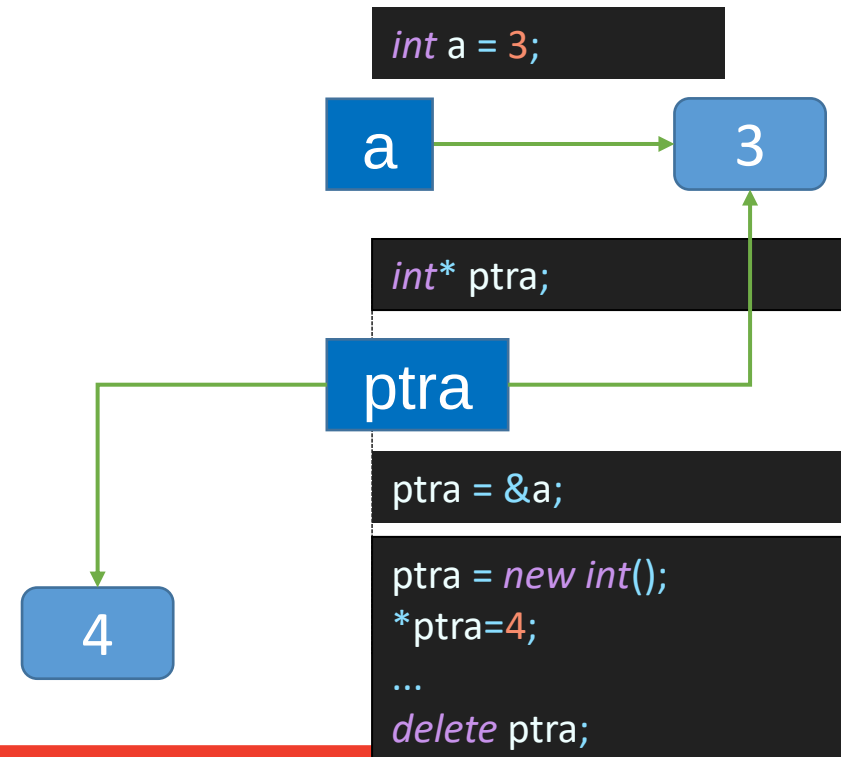
- **static** variables

- Reduces the variable's visibility to the block in which it is defined.
- A static local variable is a global variable that can only be accessed in the context of a given function.
Its lifetime corresponds to that of a global variable: its state (value) is maintained between each call to the.

Allocation of a variables and a pointer



- When you allocate a variable
 - you create **an identifier and associated memory**
- When you allocate a pointer
 - you create **an identifier only!**
 - **No memory** is associated!
 - On peut l'associer dynamiquement !
Et il faut la libérer par la suite It can be associated dynamically!
And it has to be freed afterwards.



C++ Basics, instructions



- Conditions

```
int a,b;  
if (a==3) {  
    b=2;  
} else {  
    b=6;  
}
```

```
int a,b;  
b = (a==3)?2:6;
```

```
int a,b;  
switch (a) {  
    case 3 : b=2;  
    break;  
    default : b=6;  
}
```

- Loops

```
for (int i=0; i<10; i++) {  
    ...  
}  
  
int i=10;  
while (i>0) {  
    i--;  
}
```

```
int j=0;  
do {  
    ...  
    j++;  
} while (j<10);  
  
std::vector<int> t={1,6,2,4,8,0,9,5,3,7};  
for (auto &it:t) { std::cout << it << std::endl; }
```

C++ Object-oriented programming



- Groups **data and associated treatments** under the same structure
 - Notion of classes.
- Data : **Attributes**
 - Variables describing an entity, or,
 - Constants in the context of the class
- Treatments : **Methods** (procedures, functions)
 - Internal processing on attributes,
 - Interface methods, tools for reading or writing attributes

C++ classes, encapsulation



- **Classes:** (template)
 - Encapsulation of:
 - Data, and
 - Available procedures for a given entity.
 - Data, variable, constant: **attributes**
 - Procedures, functions: **methods**
 - May be overloaded using several type of parameters
- **Objects** are instantiated based on a class.
 - Calling one of the constructors (with or without parameters)
 - Destructor is automatically called at deletion.

```
class ClassName {  
    Declaration of attributes  
    Declaration of methods  
};
```

```
ClassName object1;  
ClassName object2(...);  
object1.attribut1 = ...
```

Encapsulation, securization of data



- Protect access to data to guarantee data consistency
 - **Private attributes and methods**, by default in C++
 - Accessible only from methods of the same class
 - **Public attributes and methods**
 - Accessible directly from any object in this class.
- Secure attributes
 - **getter**: Public method used to read the value of a private attribute.
 - **setter**: Public method used to assign a valid value to a private attribute.

Simple encapsulation exemple



```
#include <iostream>

class Test {
private:
    int value=0;
    int square=0;
public:
    Test() {};
    int getValue() { return value; }
    int getSquare() { return square; }
    void setValue(int p_value) {value=p_value>5?p_value; square=p_value*p_value; }
};

int main() {
    Test t1;
    t1.setValue(3);
    std::cout << "t1.value=" << t1.getValue()
        << ", t1.square=" << t1.getSquare() << std::endl;
}
```

t1.value=3, t1.square=9

Constructors and destructor details



- Class Constructor
 - Method with same name as the class,
 - No return type.
 - With or without parameters.
 - Called
 - Statically,
 - or dynamically with the new operator.
- Class destructor
 - Method with same name as the class prefixed with ~
 - Is called for any deletion of an object
 - **End of block** where it was statically created
 - With **delete** for dynamically allocated objects

```
#include <iostream>

class Test {
    int a;
public:
    Test():a(1) {};
    Test(int p_a):a(p_a){};
    ~Test() { std::cout << "destructor: a="
                << a << std::endl; }
    int geta() { return a; }
};

int main() {
    Test t1;
    Test *t2 = new Test(3);
    std::cout << "t1.a=" << t1.geta()
                << ", t2->a=" << t2->geta()
                << std::endl;
    delete t2;
}
```

t1.a=1, t2->a=3
destructor: a=3
destructor: a=1

Initialisation of attributs



- The constructor **must** initialize all attributs
 - Inside the header,
 - Or in the body of the constructor.

```
class Test {  
public:  
    Test(int p_n , float p_m) : n(p_n), m(p_m) {};  
private:  
    int n;  
    float m;  
};
```

```
class Test {  
public:  
    Test(int p_n, float p_m) { n = p_n; m = p_m; };  
private:  
    int n;  
    float m;  
};
```

Exemple of C++ classes with Qt



```
#include <QString>
#include <QDebug>

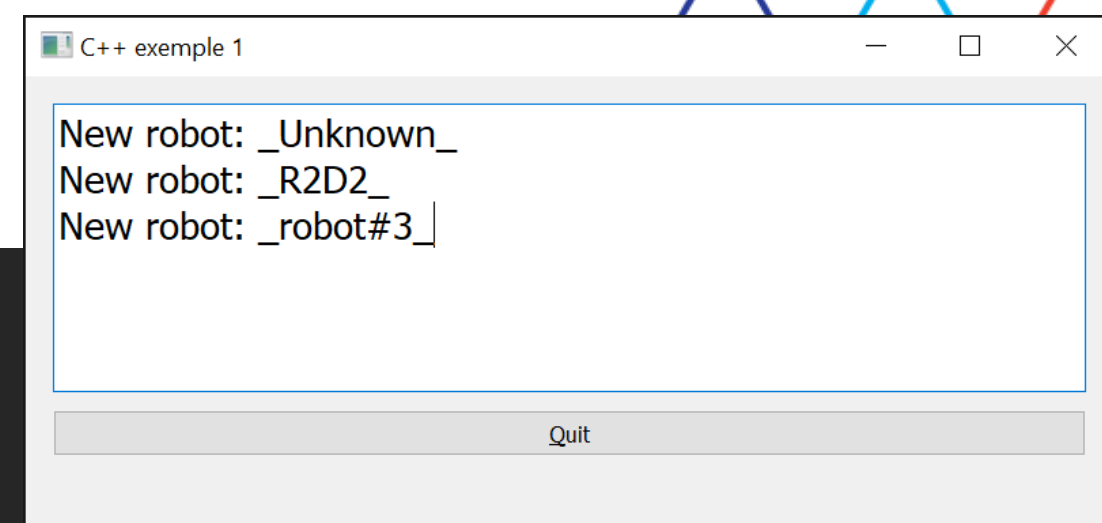
class Robot {
public:
// constructors
    Robot():name("_Unknown_") {};
    Robot(unsigned short number):name("_robot#" + QString::number(number) + "_") {};
    Robot(const QString &p_name) { setName(p_name); };
// methods
    QString getName() { return name; }
    void setName(const QString &p_name) { name = "_" + p_name + "_"; };
// destructor
    ~Robot() { qDebug() << "Delete " << name; }
private:
    QString name; // attribute
};
```

Exemple of C++ classes (2)

```
class MainWindow : public QMainWindow {
Q_OBJECT
public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
private:
    Ui::MainWindow *ui;
    Robot R1;
};

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent), ui(new Ui::MainWindow) {
    ui->setupUi(this);
    Robot R2("R2D2");
    Robot *R3=new Robot(3);

    ui->textEdit->append("New robot: "+R1.getName());
    ui->textEdit->append("New robot: "+R2.getName());
    ui->textEdit->append("New robot: "+R3->getName());
    delete R3;
}
```



Console

09:34:43: Starting F:\...\debug\basicCpp.exe ...

Delete "_robot#3_"

Calling delete R3

Delete "_R2D2_"

Local variable

Delete "_Unknown_"

When closing the app

Attributes initialisation by constructor



- Class constructor may call attribute initializers
 - More efficient than affect attributes in the body of the constructor

```
class Test {  
public:  
    Test(int p_n , float p_m) : n(p_n), m(p_m) {};  
private:  
    int n;  
    float m;  
};
```

```
class Test {  
public:  
    Test(int p_n, float p_m) { n = p_n; m = p_m; };  
private:  
    int n;  
    float m;  
};
```

Copy constructor

- When you create an object from another object of the same class
 - A **copy constructor** is called
 - Defined by default
 - Or overloaded for the class
 - The default copy constructor
 - Duplicate the whole memory of the original

```
#include <iostream>
class Test {
public:
    Test():_a(0),_b(2),_c(1) {}
    inline int getc() {return _c;}
private:
    int _a,_b,_c;
};

int main() {
    Test T;
    Test T1(T); // default
    std::cout << "T1.c=" << T1.getc() << std::endl;
    Test T2=T1; // default
    std::cout << "T2.c=" << T2.getc() << std::endl;
}
```

T1.c=1
T2.c=1

Copy constructor

- When you create an object from another object of the same class
 - A **copy constructor** is called
 - Defined by default
 - Or overloaded for the class
 - The default copy constructor
 - Duplicate the whole memory of the original

```
#include <iostream>
class Test {
public:
    Test():_a(0),_b(2),_c(1) {}
    Test(const Test& T) { std::cout << "copie" << std::endl;
        _a=T._a;
        _b=T._b;
        _c=T._c+1;
    }
    inline int getc() { return _c; }
private:
    int _a,_b,_c;
};

int main() {
    Test T;
    Test T1(T);
    std::cout << "T1.c=" << T1.getc() << std::endl;
    Test T2=T1;
    std::cout << "T2.c=" << T2.getc() << std::endl;
}
```

copie
T1.c=2
copie
T2.c=3

Passing parameters in methods



- In C++ there are 3 ways of passing parameters:
 - **By value:**
 - The parameter is a local variable initialised by the call value.
 - **By reference:**
 - The parameter represents the calling variable for the entire method.
 - **By address:** (*comes from C, avoid if possible*)
 - The behavior is the same as for passing by reference, but the address of a variable passed on the call is manipulated.

Passing parameters example



```
class Test {  
public:  
    Test() {}  
    void byValue(int v) { _a=v/2; }  
    void byReference(int &v) { v=_a*2; }  
    void byAddress(int *v) { *v= _a*2; }  
private:  
    int _a;  
};
```

```
int main() {  
    Test t;  
    t.byValue(5);  
    int x;  
    t.byReference(x);  
    t.byAddress(&x);  
}
```

Default parameter values



- You can specify a **default value** for one or more function or method **parameters**.
 - In the function or method declaration,
 - Starting **from the rightmost** parameters.

```
class Test {  
    public:  
        Test() {}  
        void sets(int a,int b=0,int c=1)  
            { _a=a; _b=b; _c=c; }  
    private:  
        int _a,_b,_c;  
};  
  
int main() {  
    Test t;  
    t.sets(5);  
    t.sets(5,2);  
    t.sets(5,3,2);  
}
```

Methods overloading



- It is possible to define several methods with the same name.
 - If the **attributes are of different numbers and/or types**
 - taking into account the default values!
 - Use it **when several methods perform the same type of processing** on different data.

« const » specifier



- "const" specifier may be affect to:
 - Methods,
 - method parameters,
 - and objects.
- If an object is allocated using the “const” prefix:
 - Its attributes cannot be modified.
 - Any attempt to modify it will result in a compilation error.
 - Only its “const” methods can be called, as they guarantee that the object will not be modified.
 - It must be initialized when at declaration
- If a method is defined as “const”,
 - It can be called by all const and non-const objects.

« const » usages



```
class Test {  
public:  
    Test(int p_n):n(p_n) {};  
  
    int getNconst() const { return n; };  
    int getN() { return n; };  
  
private:  
    int n;  
};
```

```
Test t1(1);  
qDebug() << t1.getN();  
qDebug() << t1.getNconst();
```

```
const Test t2(2);  
qDebug() << t2.getN();  
qDebug() << t2.getNconst();
```

forbidden

Méthodes et attributs « static »



- Static methods and attributes can be accessed **without creating an object**.
 - They are called from the name of the class: **className::staticObject**
 - “Static” methods do not use non-static attributes; they generally perform processing on the data in parameter (in context with the class).
 - For example, a function that computes a person's age from their date of birth (passed as parameter).
 - A ‘static’ attribute is a variable common to all objects in the same class.
 - It can be used to define a unique identifier for each object.
 - It must be initialized in the global scope of the code.

« static » attribute example



```
#include <iostream>
class Test {
public:
    static int currentId;
    Test():id(currentId) { currentId++; }
    inline int getId() { return id;}
private:
    int id;
};

int Test::currentId=1; // once in global context only
int main() {
    Test T1;
    Test T2;
    Test T3;
    std::cout << "T1.id=" << T1.getId() << "\n"
                << "T2.id=" << T2.getId() << "\n"
                << "T3.id=" << T3.getId() << std::endl;
}
```

T1.id=1
T2.id=2
T3.id=3

Enumerations « enum »



- Enum = List of integer constants
 - Same advantage as for constant attributes:
 - Avoids the use of numeric values in codes
 - Semantics of numeric constants
 - Groups constants under the same category
 - An integer value can be specified for each constant
 - By default, the first value is 0
 - A constant defaults to the value of the previous constant +1
- Can be defined inside a class

```
enum Color { red = 1, green, blue };  
Color r = blue;
```

Study this example

```
#include <iostream>
#include <cstdlib>
#include <string>

class Test {
public:
    enum Color { rouge=0, orange, jaune, vert, bleu, indigo, violet, nbColors, none };
    static const std::string colorNames[nbColors];
    static const int tabSize=10;
    Test() {
        for (int c=0; c<tabSize; c++) {
            tabColors[c] = Color(rand()%nbColors);
        }
    }
    inline int getColor(int i) { return (i>=0 && i<tabSize)?tabColors[i]:none;}
private:
    int id;
    Color tabColors[tabSize];
};

// static const attributes must be initialized here
const std::string Test::colorNames[Test::nbColors]={"Rouge","Orange","Jaune","Vert","Bleu","Indigo","Violet"};

int main() {
    Test T;

    if (T.getColor(3)!=Test::Color::none) {
        std::cout << Test::colorNames[T.getColor(3)] << std::endl;
    }
}
```

Overloading of operators



- A feature specific to C++ that makes it easier to read and write operations.
 - For example: **A,B,C** being « **Vector2D** », coding: **A += 5.0*B - (A*C) * C;**
 - Operators can be overloaded according to operand type
 - **Vector2D += Vector2D ; double * Vector2D ; Vector2D * Vector2D ; Vector2D - Vector2D**
 - Each overload must be given its own semantics
 - For example, the * operator in **Vector2D * Vector2D** represents the scalar product and returns a real number!
- Unairy operators ([], +=, -=, ...)
- Binary operators (+, -, *, /, %, ...)

Operator overloading example



```
class Vector2D {  
    public : float x,y;  
    Vector2D():x(0),y(0) {};  
    Vector2D(const Vector2D &v) {x=v.x; y=v.y; }  
    Vector2D(float p_x, float p_y) : x(p_x), y(p_y) {};  
    Vector2D operator-(const Vector2D &op2) const {  
        return Vector2D(x - op2.x, y - op2.y);  
    }  
    Vector2D operator+(const Vector2D &op2) const {  
        return Vector2D(x + op2.x, y + op2.y);  
    }  
};
```

```
Vector2D P(1, -1);  
Vector2D Q(3, -2);  
  
Vector2D PQ = Q - P;
```

Inheritance, access control



- In inheritance, **the child class inherits the (not private) properties** and functionality of the parent class.
 - Code reusability
 - Child class just add local attributes and methods.
- A virtual parent class method **may be overridden** by a derived class method.
 - “virtual” prefix in parent method and “override” suffix in child method.
 - If the method is not overloaded in the child class, then the parent method is used.
- **Pure virtual method must be overridden** by all derived class.
 - Suffix “=0” in parent class.



Access mode in inheritance

The below table summarizes the above three modes and shows the access specifier of the members of base class in the sub class when derived in public, protected and private modes:

```
class ChildClass:public ParentClass
```

Base class member Access specifier	Type of inheritance		
	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	Not accessible	Not accessible	Not accessible

Inheritance example



```
class Shapes {
public:
    Shapes(const QPointF &_position);
    virtual ~Shapes() {};
    virtual void draw(QPainter*) = 0; // pure virtual, must be overridden
    virtual bool isIn(const QPointF &) { return false; }; // returns false by default
protected:
    QPointF position; // all shapes have a position
};

class Rectangle:public Shapes {
public:
    Rectangle(const QPointF &_position,float _width,float _height,float _angle);
    virtual void draw(QPainter*) override;
    virtual bool isIn(const QPointF &) override;
protected:
    float width; float height; float angle; // data for rectangle only
};
...
```

Inheritance example (2)

```
class Circle:public Shapes {  
public:  
    Circle(const QPointF &_position,float _radius);  
    virtual void draw(QPainter*) override;  
    virtual bool isIn(const QPointF &) override;  
protected:  
    float radius; // parameter for circle only  
};
```



Example of derived operator



```
class Shapes {
public:
    Shapes(const QPointF &_position);
    virtual ~Shapes() {};
    virtual void draw(QPainter*)=0;
    virtual bool isIn(const QPointF &) { return false; };

    virtual float operator[](int i);

protected:
    QPointF position;
};
```

```
float Shapes::operator[](int i) {
    switch(i) {
        case 0: return position.x();
        case 1: return position.y();
        default : return 0.0;
    }
}

float Circle::operator[](int i) {
    switch(i) {
        case 0: case 1: return Shapes::operator[](i);
        case 2: return radius;
        default : return 0.0;
    }
}
```

Friend class and methods



- A friend class **can access private and protected members** of other class in which it is declared as friend.
- Allow to transfer public access to external functions or methods.
- Friendship is not transitive (a friend of your friend is not your friend).
- Friendship is not inherited (your friend's children are not your friends).

Friend class and methods



```
#include <iostream>
using namespace std;
class Prof;

class Student {
    int number; // is private
    friend Prof;
public:
    Student(int _number):number(_number) {};
    Student():number(-1) {};
    friend ostream& operator<<(ostream& out,
                                const Student& s);
};

ostream& operator<<(ostream& out, const Student& s) {
    return out << s.number << endl;
}
```

```
class Prof {
    int Id;
public:
    Student disciple;
    Prof(int _id):Id(_id) {
        cout << "Disciple id="
              << disciple.number << endl;
        disciple.number=_id+1;
    };
};

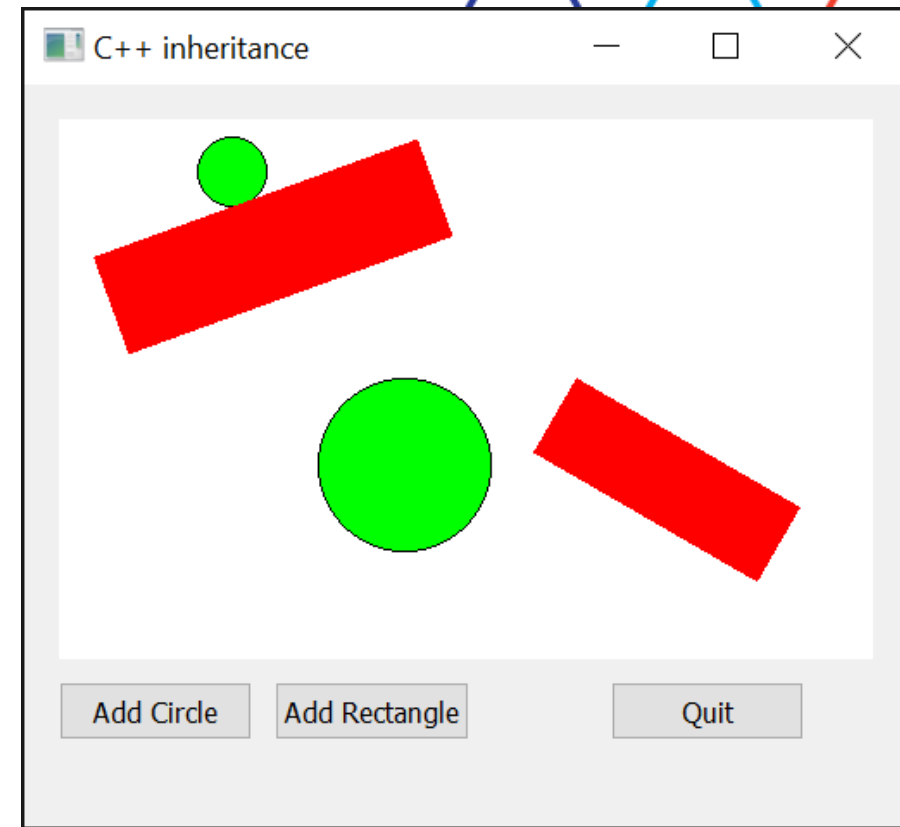
int main() {
    Prof P(1);
    cout << "Disciple id=" << P.disciple
          << endl;
    return 0;
}
```

Disciple id=-1
Disciple id=2

Polymorphism

- Goal: create heterogenous data sets
- Method:
 - Create a parent class (ex. Shapes),
 - And many derived class (ex. Circle, Rectangle...)
 - Create a set of the parent class pointers.
 - Allocate derived class in data elements.

```
QList<Shapes*> list;  
list.push_back(new Circle(QPointF(200,200),50));  
list.push_back(new Rectangle(QPointF(300,150),150,50,30));
```



Templates



- Add parameters in Classes, or functions
 - Allow to create treatment on data which type is a parameter.
- An example of function with template

```
template <class T>
void mySwap(T& a,T&b) {
    T tmp=a;
    a=b;
    b=tmp;
}
```

```
int ia=1,ib=2;
QDebug() << "before: "<< ia << "," << ib;
mySwap(ia,ib);
QDebug() << "after: "<< ia << "," << ib;

QString sa="first",sb="second";
QDebug() << "before: "<< sa << "," << sb;
mySwap(sa,sb);
QDebug() << "after: "<< sa << "," << sb;
```

Classes with template



```
template <class T>
class Element {
    T data;
    Element<T> *next;
public:
    Element(T d, Element<T> *n):data(d),next(n) {};
};

template <class T>
class Stack {
    Element<T> *top;
public:
    Stack() { top= nullptr; };
    void push(T data) { top=new Element<T>(data,top); };
};
```

```
Stack<float> floatStack;
floatStack.push(1.2f);
floatStack.push(-2.1f);
```

```
Stack<QString> stringStack;
stringStack.push("one");
stringStack.push("two");
```

STL library overview



- Usual data structures
 - array
 - vector
 - map
 - queue
 - stack
 - list
- Uses templates to embed user data.

STL Array



- Container that encapsulates fixed size data.
 - Defined in header `<array>`
`template<class T, std::size_t N> struct array;`
- Can be initialized with aggregate-initialization given at most N initializers that are convertible to T

```
#include <array>

std::array<int,5> myArray = { 1,6,2};
std::cout << "myArray[2]=" << myArray[2] << std::endl;
myArray[4] = 4;
for (auto &t:myArray) std::cout << t << std::endl;
```

```
myArray[2]=2
1
6
2
0
4
```

STL Vector



- Container that encapsulates dynamic size arrays
 - Defined in header `<vector>`
 - The storage is handled automatically, being expanded as needed.
- The elements are stored contiguously
 - Elements can be accessed using offsets to regular pointers to elements.
 - Or using iterators

```
#include <vector>
```

```
std::vector<int> myVector = {1, 6, 2};
```

```
myVector.pop_back();
```

```
myVector.push_back(7);
```

```
myVector.resize(6);
```

```
myVector[5] = 5;
```

```
std::cout << "myVector = { ";
```

```
for (auto &n : myVector) std::cout << n << ", ";
```

```
std::cout << "}; \n";
```

```
myVector = { 1, 6, 7, 0, 0, 5, };
```

STL Map



- Sorted associative container that contains key-value pairs
 - with unique keys.
 - Keys are sorted by using the comparison function Compare.
 - Defined in header `<map>`
 - Search, removal, and insertion operations have **logarithmic complexity**.
 - Maps are usually implemented as Red–black trees

```
template<class Key, class T, class Compare =std::less<Key> class map;
```

STL Map example



```
#include <iostream>
#include <map>
using namespace std;
int main() {
    // Create a map of strings to integers
    map<std::string, pair<string,string>> map;
    // Insert some values into the map
    map["UFC"] = {"University","Université de franche-Comté"};
    map["PSM"] = {"Master","Produits et Services Multimédia"};
    map["IOT"] = {"Master","Internet of Things"};
    // accede from key value
    cout << "UFC is for " << map["UFC"].second << endl;

    for (auto &it:map) {
        cout << "Key: " << it.first << ", Value: {" << it.second.first
            << "," << it.second.second << "}" << std::endl;
    }
    return 0;
}
```

UFC is for Université de franche-Comté
Key: IOT, Value: {Master,Internet of Things}
Key: PSM, Value: {Master,Produits et Services Multimédia}
Key: UFC, Value: {University,Université de franche-Comté}

Queue, stack and list...



- Queue
 - FIFO context
 - **push_back** and **pop_front**
 - **front()** to get then older element
 - **back()** to get then last pushed element
- Stack
 - LIFO context
 - **push_back** and **pop_back**
 - **back()** to get the current top element of the stack

List



- Constant time insertion and removal of elements
- from anywhere in the container.
- **insert()** to add an element before a position.

Use iterators



- Iterators are pointer to an element of a data structure
- Example for a `std::list`
 - `begin()` returns an iterator to the first element.
 - `end()` returns an iterator to the past-the-end element.
 - `operator++` moves iterator to the next element. (or `next(it)`)
 - `operator--` moves iterator to the previous element. (or `prev(it)`)
 - `operator*` returns the value of the element.
 - `operator->` returns the value of a component of the pointed class or structure.

Examples of iterators



```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<int> myList = { 2,3,1,6,8,9,5,4};

    auto it = myList.begin();
    cout << *it << endl;
    it++; it++;
    cout << *it << endl;
    it--;
    cout << *it << endl;
    return 0;
}
```

2
1
3

```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<int> myList = { 2,3,1,6,8,9,5,4};

    auto it = myList.begin();
    int s=0;
    while (it!=myList.end()) {
        s+=*it++;
    }
    cout << s << endl;
    return 0;
}
```

38

Two methods for crossing a set



- Using for loop :

```
for (auto &s:list) {  
    // traitement with the element 's'  
}
```

```
for (auto s:list) {  
    // traitement with a copy 's' of the element  
}
```

- Or using while loop:

```
auto it = list.begin();  
while (it!=list.end() && stopCondition(*it)) {  
    it++;  
}  
if (it!=list.end()) {  
    // element found  
}
```

Example of use



```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<int> myList = { 2,3,1,6,8,9,5,4};

    for (auto s:myList) {
        s++;
    }
    for (auto &s:myList) {
        cout << "|" << s;
    }
    cout << "|" << endl;
    return 0;
}
```

|2|3|1|6|8|9|5|4|

```
#include <iostream>
#include <list>
using namespace std;
int main() {
    list<int> myList = { 2,3,1,6,8,9,5,4};

    for (auto &s:myList) {
        s++;
    }
    for (auto &s:myList) {
        cout << "|" << s;
    }
    cout << "|" << endl;
    return 0;
}
```

|3|4|2|7|9|10|6|5|

Example of use



- Write a program that find the first index (i) of the myList list<int> where:
 - $\text{myList}[i] > \text{myList}[i+1]$
- Test on three different sets of values
 - { 2,3,1,6,8,9,5,4 }
 - { 2 }
 - { 2,3,6,8,9,11 }

My answer



```
#include <iostream>
#include <list>
using namespace std;
int main() {
    // case 1: list<int> myList = { 2,3,1,6,8,9,5,4};
    list<int> myList = { 2}; // case 2
    // case 3: list<int> myList = { 2,3,6,8,9,11};
    auto it = myList.begin();
    unsigned int i=0;
    while (it != myList.end() && next(it)!=myList.end() && *it<=*next(it)) {
        it++;
        i++;
    }
    if (it != myList.end() && next(it)!=myList.end()) {
        cout << "found: myList[" << i << "]= "
             << *it << " > myList[" << i+1 << "]= " << *next(it) << endl;
    } else {
        cout << "not found!" << endl;
    }
    return 0;
}
```

Ask chatGPT



```
#include <iostream>
#include <list>
int main() {
    std::list<int> myList = {5, 8, 2, 10, 7, 4, 6};
    int i = 0;
    for (auto it = myList.begin(); it != std::prev(myList.end()); ++it, ++i) {
        if (*it > *std::next(it)) {
            std::cout << "Found at index i = " << i << std::endl;
            break;
        }
    }
    return 0;
}
```



Comment your programs

Create a report using Doxygen

Comment rules



- File
 - **@author** and **@date**
 - **@todo** what to do for this file
- Class
 - Header : **@brief** goal of the class
 - Attributes : **///** meaning of the attribut
 - Methods:
 - **@brief** goal of the method
 - **@return** result of the method
 - **@param** name and meaning of the parameter
 - option: **@warning** important remarks

File header

```
/**
 * @author Benoît Piranda
 * @date 7/11/2024
 * @todo Make more tests of the triangulation algorithm with complex shapes.
 */
```

```
#ifndef MYPOLYGON_H
#define MYPOLYGON_H
#include "vector2d.h"
#include <QPainter>
#include <QDebug>
```

Class header

```
/**
 * @brief The Triangle class store 3 pointers to existing Vector2D vertices.
 * It is used by the MyPolygon class to create a set of internal triangles.
 */
```

```
class Triangle {
public:
```

Attribute

```
    Vector2D *tabPts[3]; ///< array of 3 pointers to point
```

Method header

```
/**
 * @brief Constuctor of triangle with pointers to vertices,
 * @warning The order of vertices must be CCW
 * @param p_p0 P0
 * @param p_p1 P1
 * @param p_p2 P2
 */
```

```
Triangle(Vector2D *p_p0, Vector2D *p_p1, Vector2D *p_p2) {
    tabPts[0]=p_p0;
    tabPts[1]=p_p1;
    tabPts[2]=p_p2;
}
```

Triangle Class Reference

The **Triangle** class store 3 pointers to existing **Vector2D** vertices. It is used by the **MyPolygon** class to create a set of internal triangles. [More...](#)

```
#include <mypolygon.h>
```

Public Member Functions

Triangle (**Vector2D** *p_p0, **Vector2D** *p_p1, **Vector2D** *p_p2)

Constuctor of triangle with pointers to vertices,.

bool **isInside** (const **Vector2D** &P) const

Check if a point P is inside the triangle.

double **surface** () const

Compute the surface of the triangle from vertex coordinates.

Public Attributes

Vector2D * **tabPts** [3]

array of 3 pointers to point

Private Member Functions

bool **isOnTheLeft** (const **Vector2D** &P, int iA, int iB) const

Tool to detect if a point is inside a triangle.

◆ Triangle()

```
Triangle::Triangle ( Vector2D * p_p0,  
                    Vector2D * p_p1,  
                    Vector2D * p_p2 )
```

inline

Constructor of triangle with pointers to vertices,.

Warning

The order of vertices must be CCW

Parameters

p_p0 P0

p_p1 P1

p_p2 P2

Definition at line **27** of file **mypolygon.h**.

Configuration of Doxygen



- Doxygen website: <https://www.doxygen.nl>
- Install for your system
- Run doxywizard

Specify the working directory from which doxygen will run

C:/Program Files/doxygen

Select...

Configure doxygen using the Wizard and/or Expert tab, then switch to the Run tab to generate the documentation

Wizard

Expert

Run

Topics

Project

Mode

Output

Diagrams

Provide some information about the project you are documenting

Project name: QtPolygons

Project synopsis: Draw polygons in a Qt window

Project version or id: 1.0

Project logo: Select...

No Project logo selected.

Specify the directory to scan for source code

Source code directory: C:/Users/bpiranda/Downloads/QtPolygonsExercise

Select...

☐ Scan recursively

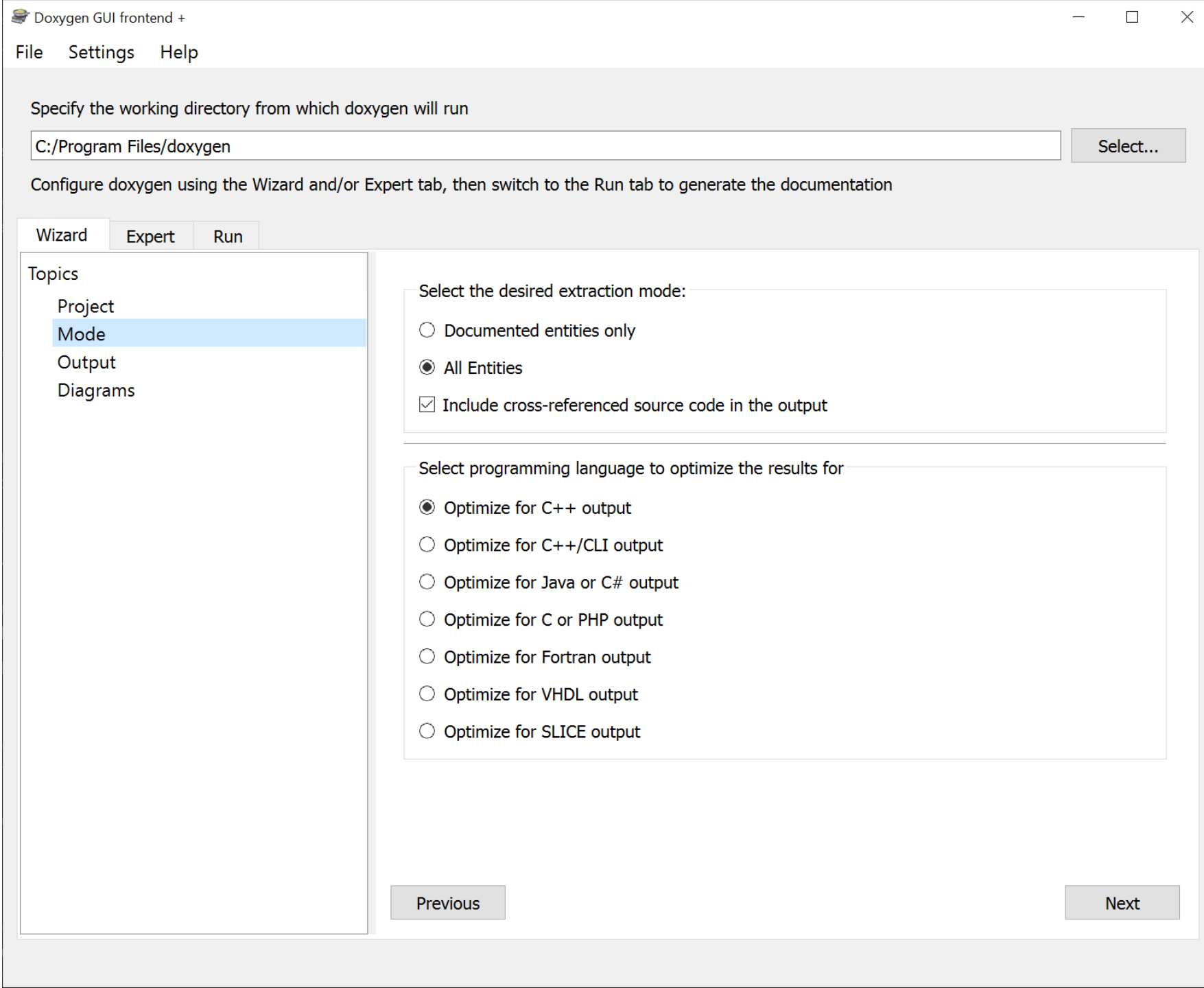
Specify the directory where doxygen should put the generated documentation

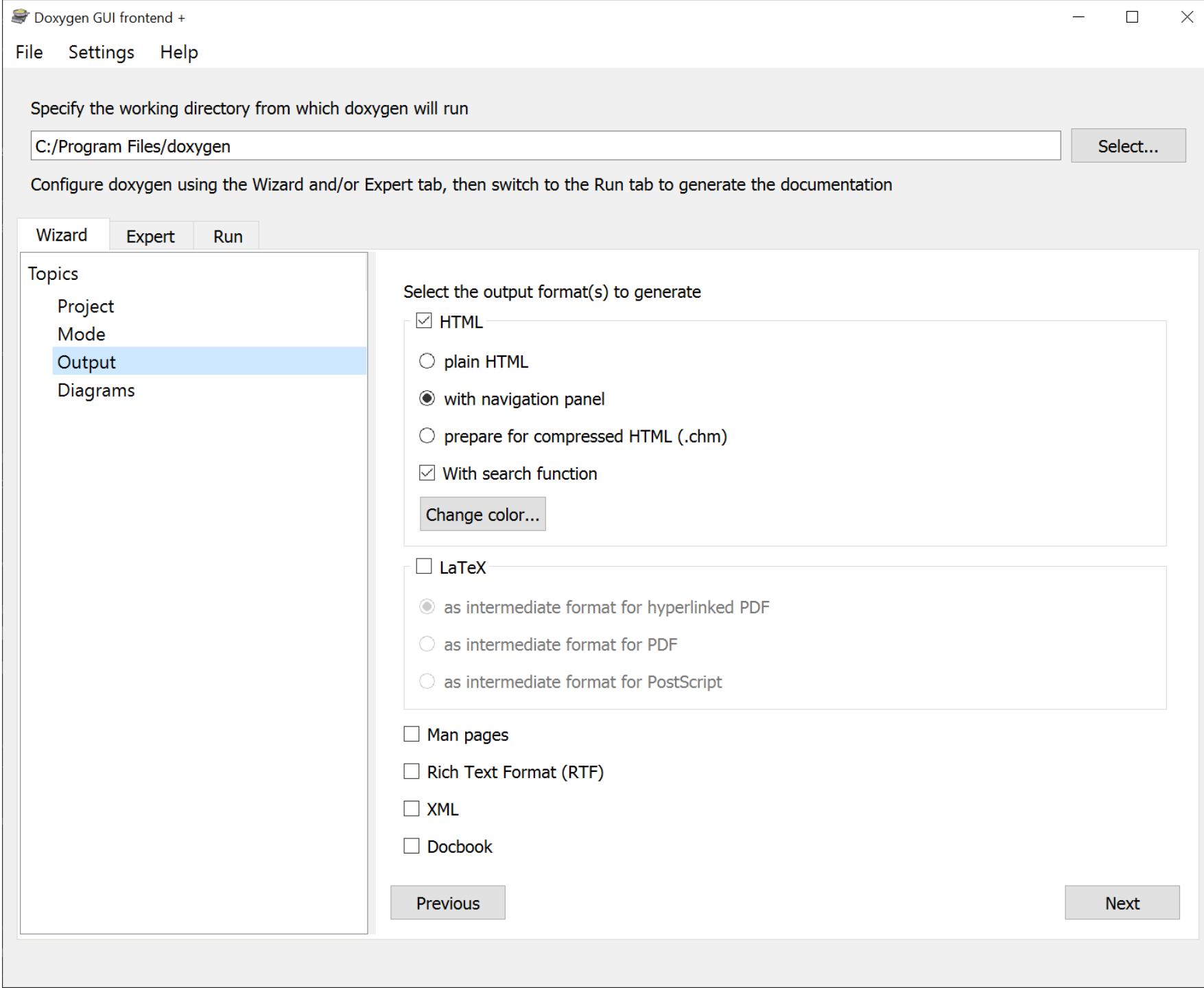
Destination directory: C:/Users/bpiranda/Downloads/QtPolygonsExercise/Doxygen

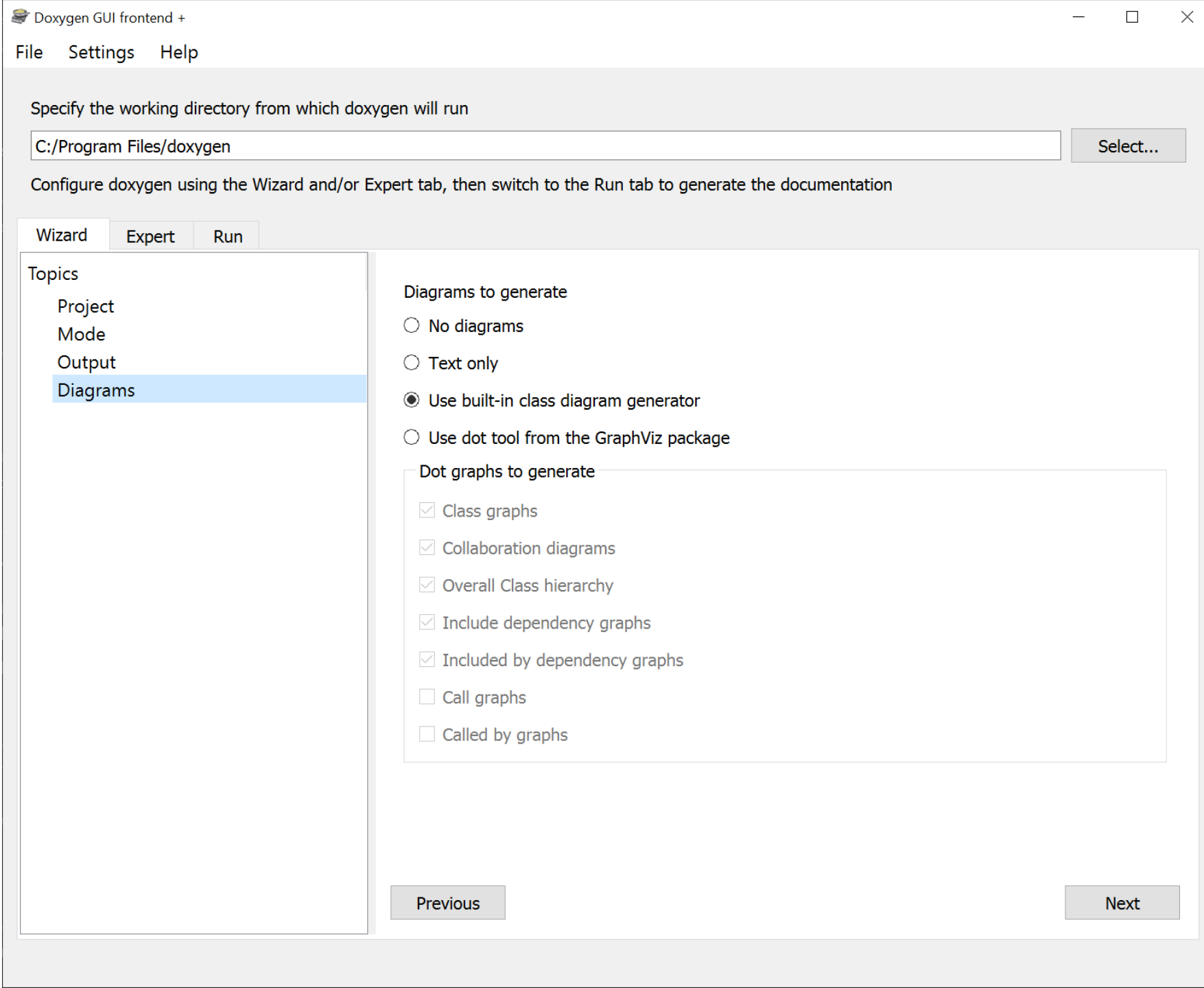
Select...

Previous

Next







Expert setup



- For Qt projects check QT_AUTOBRIEF (Project)
- Build, check:
 - EXTRACT_ALL, EXTRACT_PRIVATE, EXTRACT_PRIV_VIRTUAL,
 - EXTRACT_STATIC, EXTRACT_LOCAL_METHODS
- Source Browser, check
 - SOURCE_BROWSER (to have access to the source from the doc)

